

Vertex AI：使用自動封裝功能，在 Vertex AI Training 透過 Hugging Face 微調 BERT

來源：<https://codelabs.developers.google.com/vertex-training-autopkg?hl=zh-tw>

步驟數：8

目錄

1. [1. 總覽](#)
2. [2. 用途總覽](#)
3. [3. Vertex AI 簡介](#)
4. [4. 設定環境](#)
5. [5. 編寫訓練程式碼](#)
6. [6. 在本機進行容器化及執行訓練程式碼](#)
7. [7. 建立自訂工作](#)
8. [8. 清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何使用自動封裝功能，在 Vertex AI 訓練 上執行自訂訓練工作。Vertex AI 的自訂訓練工作會使用容器。如果您不想自行建構映像檔，可以使用自動封裝功能，根據程式碼建構自訂 Docker 映像檔、將映像檔推送至 Container Registry，並根據映像檔啟動 CustomJob。

課程內容

內容如下：

- 使用[本機模式](#)測試程式碼。
- 使用[自動封裝](#)功能設定及啟動自訂訓練工作。

在 Google Cloud 上執行這個實驗室的總費用約為 **\$2 美元**。

步驟 2 · 約 1 分鐘

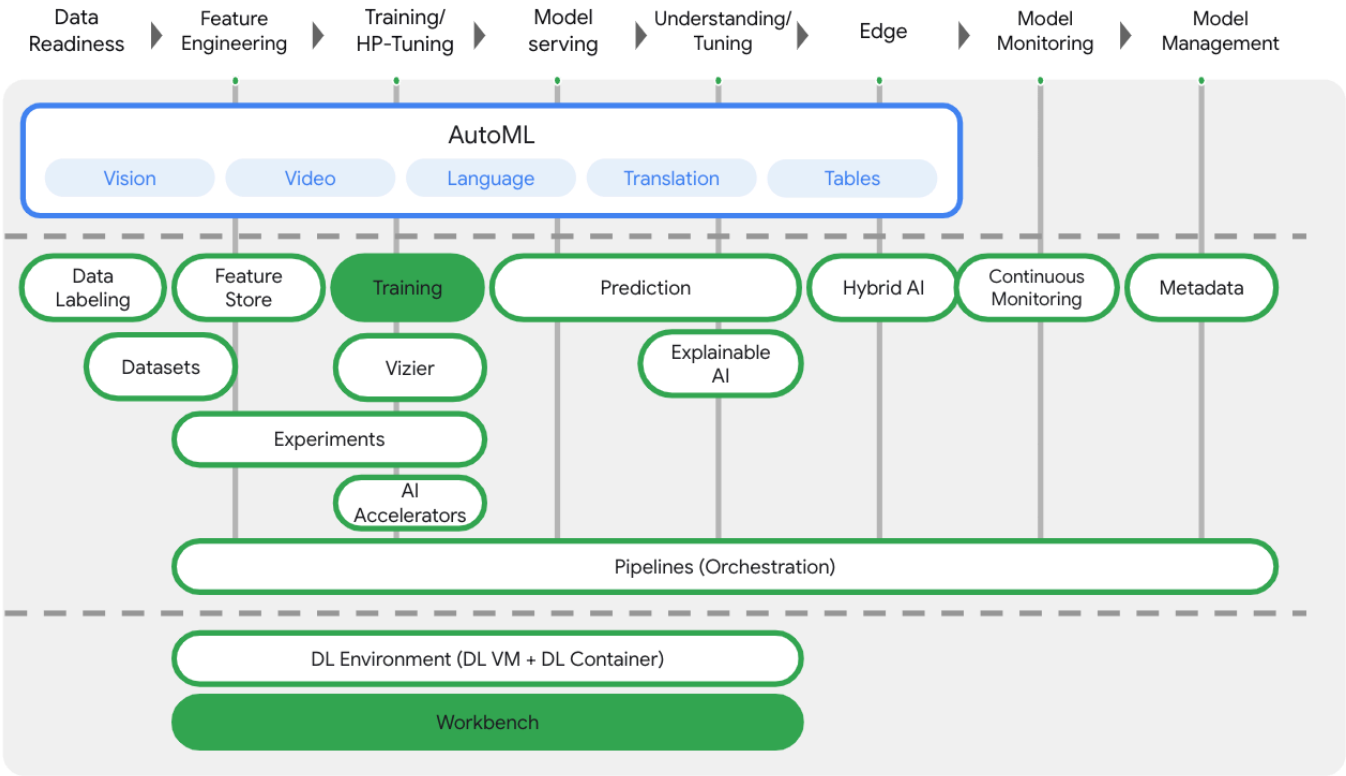
2. 用途總覽

您將使用 [Hugging Face](#) 的程式庫，根據 [IMDB 資料集](#) 微調 Bert 模型。模型會預測電影評論是正面還是負面。資料集會從 [Hugging Face 資料集程式庫](#) 下載，而 Bert 模型則會從 [Hugging Face Transformers 程式庫](#) 下載。

3. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱[支援頁面](#)。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室的重點是訓練和 Workbench。



步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

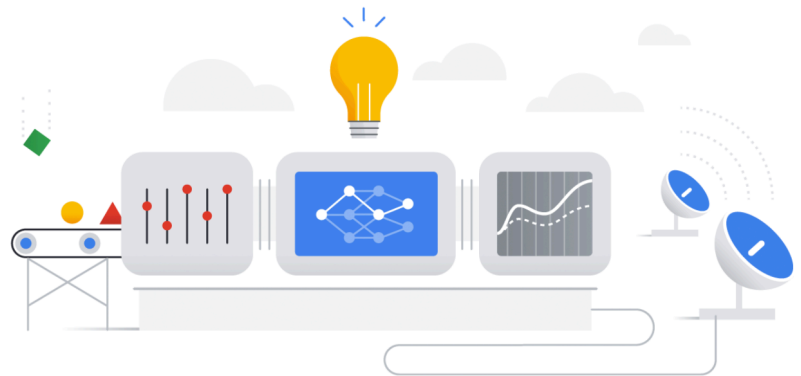
步驟 2：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

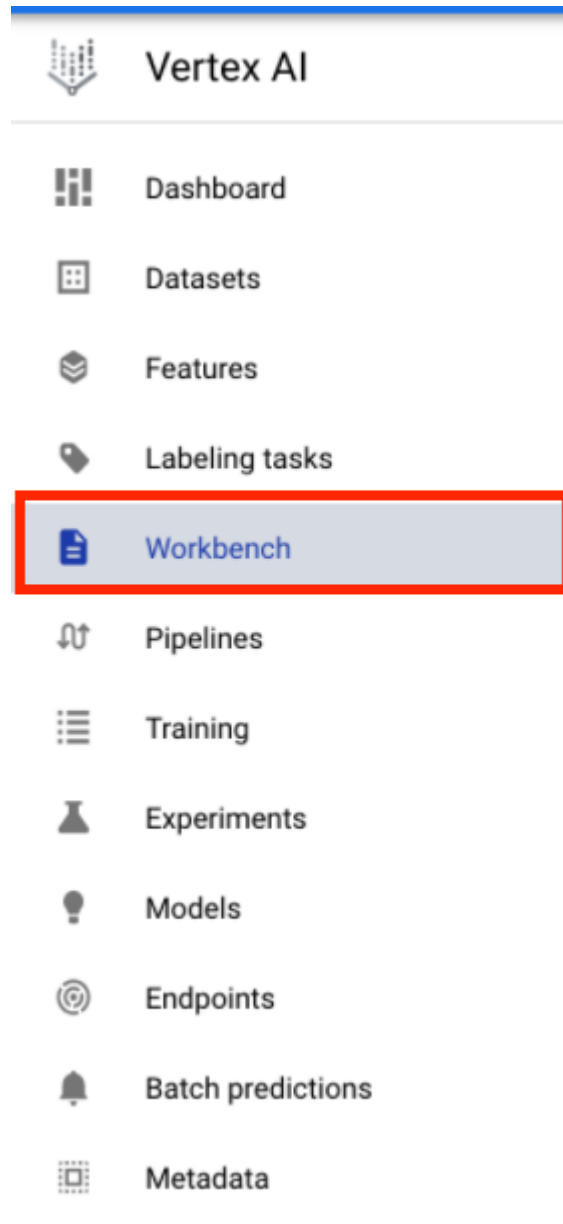
+ CREATE BATCH PREDICTION

步驟 3：啟用 Container Registry API

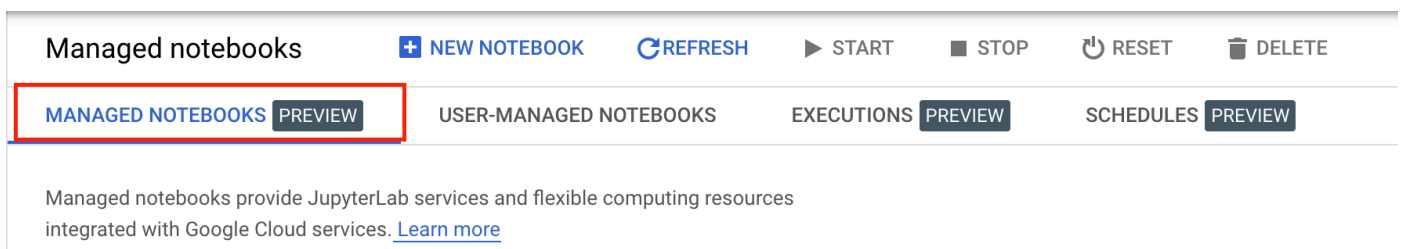
前往「Container Registry」，然後選取「啟用」（如果尚未啟用）。您將使用這個工具為自訂訓練工作建立容器。

步驟 4：建立 Vertex AI Workbench 執行個體

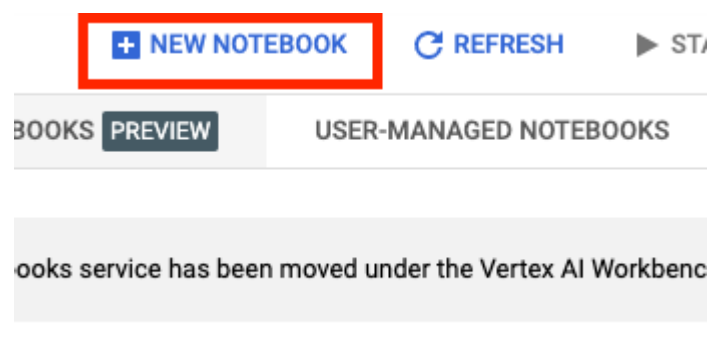
在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



然後按一下「MANAGED NOTEBOOKS」(代管型筆記本)：



然後選取「新增記事本」。



為筆記本命名，然後按一下「進階設定」。

← Create a managed notebook

Notebook name *

my-notebook

Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.
Name must be 63 characters or fewer, must start with a letter and include only lowercase letters, digits or '-'.

Region *

us-central1 (Iowa)



Advanced settings



在「進階設定」下方啟用閒置關機功能，並將分鐘數設為 60。也就是說，筆記本會在閒置時自動關機，避免產生不必要的費用。

Idle shutdown

☒ Enable Idle Shutdown

Time of inactivity before shutdown (Minutes) *

60

Must be integer: 1-600

其他進階設定可以保留原樣。

接著點選「建立」。

建立執行個體後，請選取「Open JupyterLab」。



my-notebook

OPEN JUPYTERLAB

首次使用新執行個體時，系統會要求您驗證身分。

Authenticate your managed notebook

To use managed notebooks with your Google Cloud Platform data, you need to grant the Google Cloud SDK permission to access your data and authenticate your managed notebook.

Click the button below to get your authentication code.

Get authentication code 

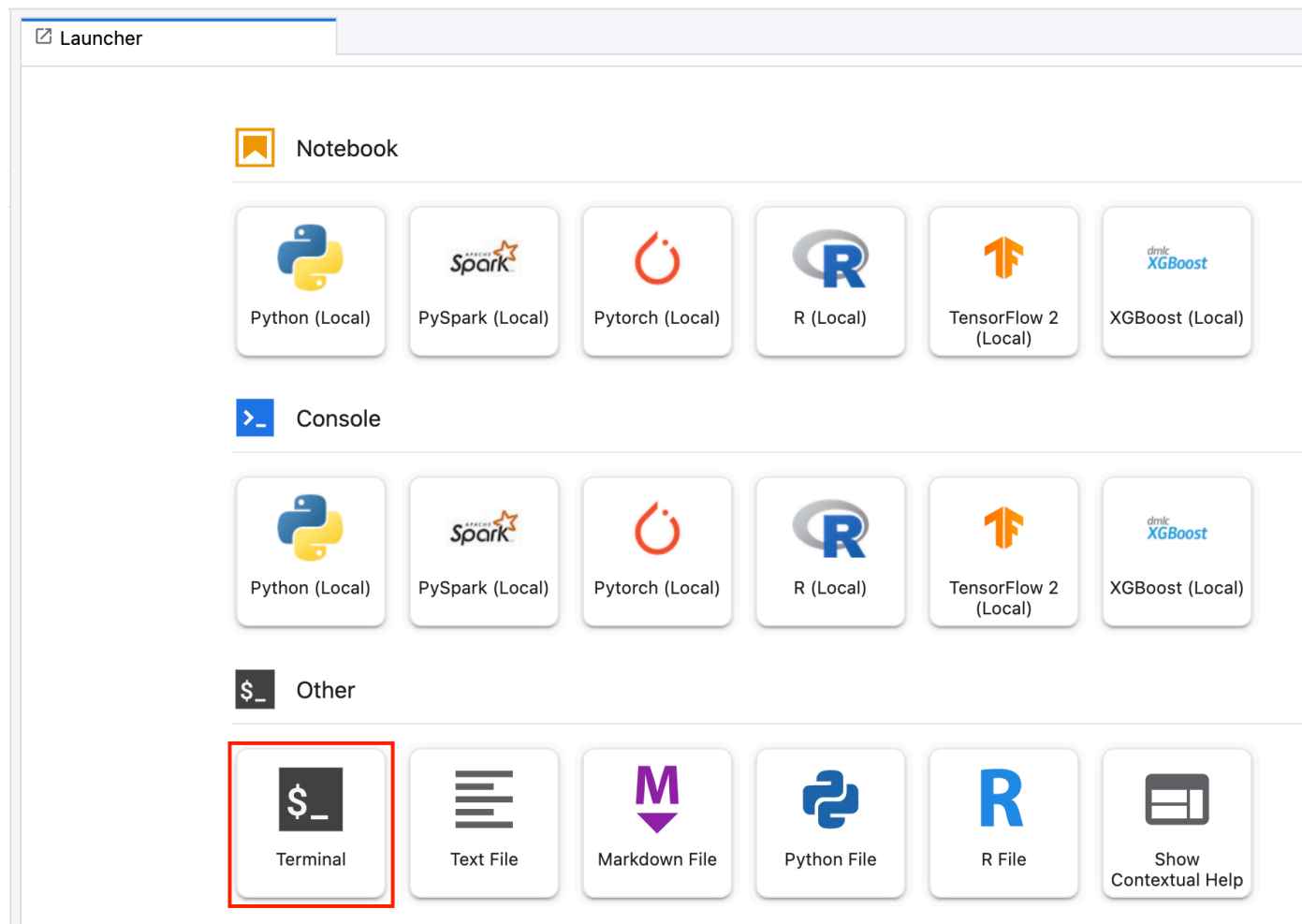
EXIT

AUTHENTICATE

步驟 5 · 約 10 分鐘

5. 編寫訓練程式碼

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



建立名為 `autopkg-codelab` 的新目錄，然後 `cd` 到該目錄。

```
mkdir autopkg-codelab
cd autopkg-codelab
```

在終端機中執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案：

```
mkdir trainer
touch trainer/task.py
```

`autopkg-codelab/` 目錄現在應該包含下列項目：

```
+ trainer/
  + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

```

import argparse

import tensorflow as tf
from datasets import load_dataset
from transformers import AutoTokenizer
from transformers import TFAutoModelForSequenceClassification

CHECKPOINT = "bert-base-cased"

def get_args():
    '''Parses args.'''

    parser = argparse.ArgumentParser()
    parser.add_argument(
        '--epochs',
        required=False,
        default=3,
        type=int,
        help='number of epochs')
    parser.add_argument(
        '--job_dir',
        required=True,
        type=str,
        help='bucket to store saved model, include gs://')
    args = parser.parse_args()
    return args

def create_datasets():
    '''Creates a tf.data.Dataset for train and evaluation.'''

    raw_datasets = load_dataset('imdb')
    tokenizer = AutoTokenizer.from_pretrained(CHECKPOINT)
    tokenized_datasets = raw_datasets.map((lambda examples: tokenize_function(examples,
tokenizer)), batched=True)

    # To speed up training, we use only a portion of the data.
    # Use full_train_dataset and full_eval_dataset if you want to train on all the data.
    small_train_dataset = tokenized_datasets['train'].shuffle(seed=42).select(range(1000))
    small_eval_dataset = tokenized_datasets['test'].shuffle(seed=42).select(range(1000))
    full_train_dataset = tokenized_datasets['train']
    full_eval_dataset = tokenized_datasets['test']

    tf_train_dataset = small_train_dataset.remove_columns(['text']).with_format("tensorflow")
    tf_eval_dataset = small_eval_dataset.remove_columns(['text']).with_format("tensorflow")

    train_features = {x: tf_train_dataset[x] for x in tokenizer.model_input_names}
    train_tf_dataset = tf.data.Dataset.from_tensor_slices((train_features,
tf_train_dataset["label"]))
    train_tf_dataset = train_tf_dataset.shuffle(len(tf_train_dataset)).batch(8)

    eval_features = {x: tf_eval_dataset[x] for x in tokenizer.model_input_names}

```

```

    eval_tf_dataset = tf.data.Dataset.from_tensor_slices((eval_features,
tf_eval_dataset["label"]))
    eval_tf_dataset = eval_tf_dataset.batch(8)

    return train_tf_dataset, eval_tf_dataset

def tokenize_function(examples, tokenizer):
    '''Tokenizes text examples.'''

    return tokenizer(examples['text'], padding='max_length', truncation=True)

def main():
    args = get_args()
    train_tf_dataset, eval_tf_dataset = create_datasets()
    model = TFAutoModelForSequenceClassification.from_pretrained(CHECKPOINT, num_labels=2)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=tf.metrics.SparseCategoricalAccuracy(),
    )

    model.fit(train_tf_dataset, validation_data=eval_tf_dataset, epochs=args.epochs)
    model.save(f'{args.job_dir}/model_output')

if __name__ == "__main__":
    main()

```

請注意以下幾點：

- `CHECKPOINT` 是要微調的模型。在本例中，我們使用 Bert。
- `TFAutoModelForSequenceClassification` 方法會在 TensorFlow 中載入指定的語言模型架構和權重，並在頂端新增分類標題，以及隨機初始化的權重。在本例中，我們有二元分類問題 (正面或負面)，因此我們為這個分類器指定 `num_labels=2`。

6. 在本機進行容器化及執行訓練程式碼

您可以使用 `gcloud ai custom-jobs local-run` 指令，根據訓練程式碼建構 Docker 容器映像檔，並在本機電腦上以容器形式執行該映像檔。在本機執行容器時，訓練程式碼的執行方式與在 Vertex AI 訓練上執行時類似，有助於您在 Vertex AI 中執行自訂訓練工作前，偵錯程式碼問題。

在訓練工作中，我們會將訓練好的模型匯出至 Cloud Storage 值區。在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

接著建立值區。如果你有現有值區，也可以使用該值區。

```
BUCKET_NAME="gs://${PROJECT_ID}-bucket"
gsutil mb -l us-central1 $BUCKET_NAME
```

在 Vertex AI 訓練 上執行自訂訓練工作時，我們會使用 GPU。但由於我們並未指定使用 GPU 的 Workbench 執行個體，因此會使用以 CPU 為基礎的映像檔進行本機測試。在本範例中，我們使用 [Vertex AI 訓練預建容器](#)。

執行下列指令，將 Docker 映像檔的 URI 設為容器的基礎。

```
BASE_CPU_IMAGE=us-docker.pkg.dev/vertex-ai/training/tf-cpu.2-7:latest
```

然後為本機執行指令建構的 Docker 映像檔設定名稱。

```
OUTPUT_IMAGE=$PROJECT_ID-local-package-cpu:latest
```

我們的訓練程式碼使用 Hugging Face 資料集和轉換器程式庫。這些程式庫未包含在我們選取的基本映像檔中，因此我們需要將其做為必要條件提供。為此，我們會在 `autopkg-codelab` 目錄中建立 `requirements.txt` 檔案。

確認您位於 `autopkg-codelab` 目錄，然後在終端機中輸入下列內容。

```
touch requirements.txt
```

`autopkg-codelab` 目錄現在應該包含下列項目：

```
+ requirements.txt
+ trainer/
  + task.py
```

開啟需求檔案，並貼上下列內容

```
datasets==1.18.2
transformers==4.16.2
```

最後，執行 `gcloud ai custom-jobs local-run` 指令，在 Workbench 代管執行個體上啟動訓練。

```
gcloud ai custom-jobs local-run \
--executor-image-uri=$BASE_CPU_IMAGE \
--python-module=trainer.task \
--output-image-uri=$OUTPUT_IMAGE \
-- \
--job_dir=$BUCKET_NAME
```

您應該會看到 Docker 映像檔正在建構。我們新增至 `requirements.txt` 檔案的依附元件會透過 `pip` 安裝。首次執行這項指令時，可能需要幾分鐘才能完成。建構完映像檔後，`task.py` 檔案就會開始執行，您會看到模型訓練作業。畫面應如下所示：

```
876ba43f19faee861df134affd7a3f60fc38a1. Subsequent calls will reuse this data.
100%|██████████| 3/3 [00:00<00:00, 611.74it/s]
Downloading: 100%|██████████| 29.0/29.0 [00:00<00:00, 18.4kB/s]
Downloading: 100%|██████████| 570/570 [00:00<00:00, 329kB/s]
Downloading: 100%|██████████| 208k/208k [00:00<00:00, 1.76MB/s]
Downloading: 100%|██████████| 426k/426k [00:00<00:00, 3.83MB/s]
100%|██████████| 25/25 [00:14<00:00, 1.69ba/s]
100%|██████████| 25/25 [00:14<00:00, 1.73ba/s]
100%|██████████| 50/50 [00:28<00:00, 1.73ba/s]
Downloading: 100%|██████████| 502M/502M [00:11<00:00, 44.0MB/s]
All model checkpoint layers were used when initializing TFBertForSequenceClassification.

Some layers of TFBertForSequenceClassification were not initialized from the model checkpoint at bert-base-cased and are newly initialized: ['classifier']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Epoch 1/3
 5/125 [>.....] - ETA: 47:40 - loss: 3.6180 - sparse_categorical_accuracy: 0.5000
```

由於我們並未在本機使用 GPU，模型訓練需要很長時間。您可以按下 `Ctrl+c` 取消本機訓練，不必等待工作完成。

請注意，如要進一步測試，您也可以直接執行上述建構的映像檔，不必重新封裝。

```
gcloud beta ai custom-jobs local-run \
--executor-image-uri=$OUTPUT_IMAGE \
-- \
--job_dir=$BUCKET_NAME \
--epochs=1
```

7. 建立自訂工作

我們已測試過本機模式，現在要使用自動封裝功能，在 Vertex AI 訓練啟動自訂訓練工作。這項功能會透過單一指令執行下列操作：

- 根據程式碼建構自訂 Docker 映像檔。
- 將映像檔推送至 Container Registry。
- 根據圖片發起 `CustomJob`。

返回終端機，然後將 `cd` 向上移至 `autopkg-codelab` 目錄上方一層。

```
+ autopkg-codelab
+ requirements.txt
+ trainer/
+ task.py
```

將 Vertex AI 訓練預建的 TensorFlow GPU 映像檔指定為自訂訓練工作的基礎映像檔。

```
BASE_GPU_IMAGE=us-docker.pkg.dev/vertex-ai/training/tf-gpu.2-7:latest
```

接著，執行 `gcloud ai custom-jobs create` 指令。首先，這項指令會根據訓練程式碼建構自訂 Docker 映像檔。基礎映像檔是我們設為 `BASE_GPU_IMAGE` 的 Vertex AI 訓練預建容器。接著，自動封裝功能會根據 `requirements.txt` 檔案中的指定項目，透過 `pip` 安裝資料集和轉換器程式庫。

```
gcloud ai custom-jobs create \
--region=us-central1 \
--display-name=fine_tune_bert \
--args=--job_dir=$BUCKET_NAME \
--worker-pool-spec=machine-type=n1-standard-4,replica-count=1,accelerator-
type=NVIDIA_TESLA_V100,executor-image-uri=$BASE_GPU_IMAGE,local-package-path=autopkg-
codelab,python-module=trainer.task
```

讓我們來看看 `worker-pool-spec` 引數。這會定義自訂工作所用的工作站集區設定。您可以指定多個工作站集區規格，建立具有多個工作站集區的自訂工作，以進行分散式訓練。在本範例中，我們只指定一個工作站集區，因為訓練程式碼未設定為分散式訓練。

這項規格的主要欄位包括：

- `machine-type` (必要)：機器類型。如要查看支援的類型，請[按這裡](#)。
- `replica-count`：這個工作站集區要使用的 worker 副本數量，預設值為 1。
- `accelerator-type`：GPU 類型。如要查看支援的類型，請[按這裡](#)。在本範例中，我們指定了一個 NVIDIA Tesla V100 GPU。
- `accelerator-count`：工作站集區中每個 VM 要使用的 GPU 數量，預設值為 1。
- `executor-image-uri`：用於執行所提供套件的容器映像檔 URI。這會設為基本映像檔。
- `local-package-path`：包含訓練程式碼的資料夾本機路徑。
- `python-module`：要在提供的套件中執行的 Python 模組名稱。

與執行本機指令時類似，您會看到 Docker 映像檔正在建構，然後訓練工作開始執行。不過，您不會看到訓練工作的輸出內容，而是看到下列訊息，確認訓練工作已啟動。請注意，首次執行 `custom-jobs create` 指令時，系統可能需要幾分鐘才能建構及推送映像檔。

```
CustomJob [projects/[REDACTED]/locations/us-central1/customJobs/2625900948454637568] is submitted successfully.

Your job is still active. You may view the status of your job with the command

$ gcloud ai custom-jobs describe projects/[REDACTED]/locations/us-central1/customJobs/2625900948454637568

or continue streaming the logs with the command

$ gcloud ai custom-jobs stream-logs projects/[REDACTED]/locations/us-central1/customJobs/2625900948454637568
```

返回 Cloud 控制台的 Vertex AI 訓練 區段，您應該會在「CUSTOM JOBS」(自訂工作) 下方看到正在執行的工作。

Vertex AI

Dashboard

Datasets

Features

Labelling tasks

Workbench

Pipelines

Training

Experiments

Models

Training + CREATE

TRAINING PIPELINES **CUSTOM JOBS** HYPERPARAMETER TUNING JOBS

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models.
[Learn more](#)

Region
us-central1 (Iowa)

Filter

 Enter a property name

Name	ID	Status	Job type
fine_tune_bert	2625900948454637568	🔄 Pending	Custom job

這項工作大約需要 20 分鐘才能完成。

完成後，您應該會在 bucket 的 `model_output` 目錄中看到下列已儲存的模型構件。

UPLOAD FILES UPLOAD FOLDER CREATE FOLDER MANAGE HOLDS DOWNLOAD DELETE

Filter by name prefix only ▼

Filter Filter objects and folders

☐	Name	Size	Type	Created ?	Storage class	Last modified	Public access ?
☐	assets/	—	Folder	—	—	—	—
☐	keras_metadata.pb	141.3 KB		2 Feb 202...	Standard	2 Feb 2022...	Not public
☐	saved_model.pb	7.1 MB		2 Feb 202...	Standard	2 Feb 2022...	Not public
☐	variables/	—	Folder	—	—	—	—

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

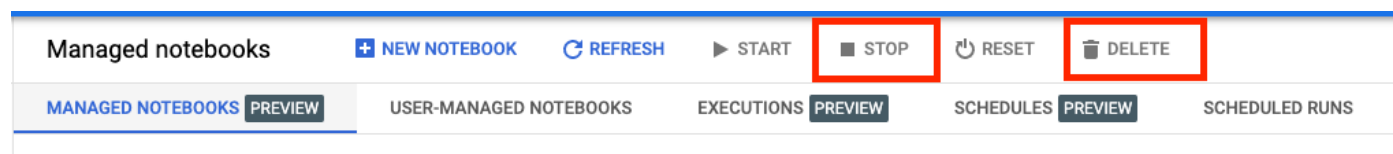
- 在本機進行容器化及執行訓練程式碼
- 使用自動封裝功能，將訓練工作提交至 Vertex AI 訓練

如要進一步瞭解 Vertex AI 的其他部分，請參閱[說明文件](#)。

步驟 8 · 約 1 分鐘

8. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

